

ffmpeg

Some Introduction

Term	Defination
Video Stream	a sequence of compressed image frame
Audio Stream	compressed audio data
Container	The "wrapper" format, how streams are organized in the file
Codec	The algorithm used to compress/decompress the actual video or audio data

The Container vs Codec distinction

Term	What it is?	Examples
Container	Wrapper format	mp4 , .mkv , .avi , .webm
Codec	algorithm	H.264 , H.265 , VP9 , AAC , MP3 , Opus

A .mp4 file can contain video encoded with H.264 or H.265. An .mkv file can contain almost anything. The container and the codec are independent — this is why "just changing the extension" does absolutely nothing.

Bitrate

- **bitrate** :- A bitrate is the amount of data used to store or transmit audio/video per second, usually measured in kbps (kilobits per second) or Mbps (megabits per second).

higher bitrate usually means

- Better image quality
- Fewer compression artifacts
- Larger file size

Relationship Between Bitrate and File size

rough formula

Why not high bitrate?

```
Higher Bitrate
  ↓
Larger File
  ↓
More Storage
  ↓
More Bandwidth
```

This won't be easy for a streaming services

Bitrate vs Resolution

Resolution means

- how many pixels exists

Bitrate

- how much data we have to describe these pixels

Imagine you're describing a beautiful landscape to someone.

1. Low bitrate

"There is a tree."

Very little information.

2. High bitrate

"There is a large oak tree with green leaves, a river behind it, mountains in the distance ... "

Much more information.

Quantizer

A quantizer (QP = Quantization Parameter) controls how aggressively a video encoder throws away detail during compression.

Lower quantizer = higher quality and larger files;

higher quantizer = lower quality and smaller files.

Quantization is one of the main reasons modern video compression works.

Initial	—(Quantizer)—→	final
102		100
103		100
104		100
105		100
106		100
We lost details		We compressed the file.

Why compression exists at all?

- The reason is still the same the redundancy.
- For ex. :- raw, uncompressed video is absurdly large. A single second of 1080p video at 30fps, uncompressed, is

roughly 1.5 GB. A 2-hour movie would be ~10 TB.

What is the thing we leverage in this example?

1. **Spatial redundancy** — within a single frame, large areas are similar (a blue sky). No need to store every pixel independently.
2. **Temporal redundancy** — between frames, most of the image doesn't change. Only the differences need to be stored.

Whata is FFMPEG?

- it is a CLI tool, that can do the following.
1. Decode any stream from a container using the appropriate codec
 2. Re-encode it with a different codec
 3. Re-wrap it into a different container
 4. Filter/process it in between (resize, trim, merge, etc.)

General pipeline

```
[Input File] → Demux → Decode → Filter → Encode →  
Mux → [Output File]
```

- Demux = unpack streams from the container
- Decode = decompress the raw data
- Filter = optional processing (resize, crop, etc.)
- Encode = recompress with a new codec
- Mux = pack into the new container

How other services like PornHub, Netflix, YouTube etc use this?

1. File gets decoded
2. Re-encode into different version -> Different resolutions
3. Served adaptively based on the connection speed.

this is called **ABR** — Adaptive Bitrate Streaming

Starting the main Course:

ffprobe

- Before converting or compressing what you do?
 - Inspecting
 - We need to inspect the file we are working with

```
ffprobe recording_2026-04-29_23.53.40.mp4
```

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from  
'recording_2026-04-29_23.53.40.mp4':
```

```
Metadata:
```

```
major_brand      : isom
```

```
minor_version    : 512
```

```
compatible_brands: isomiso2avc1mp41
```

```
encoder          : Lavf62.12.100
```

```
Duration: 00:00:53.33, start: 0.000000, bitrate:  
13503 kb/s
```

```
Stream #0:0[0x1](und): Video: h264 (High) (avc1  
/ 0x31637661), yuv420p(progressive), 2560x1600,  
13477 kb/s, 39.41 fps, 330 tbr, 90k tbn (default)
```

```
Metadata:
```

```
handler_name      : VideoHandler
Stream #0:1[0x2](und): Audio: aac (LC) (mp4a /
0x6134706D), 48000 Hz, stereo, fltp, 41 kb/s
(default)
Metadata:
  handler_name      : SoundHandler
```

- The actual output will be a bit larger as it will also include the **ffprobe** information like the compilation flags, build, version
- Focus on line **Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'recording_2026-04-29_23.53.40.mp4' :**
 - **#0** this means this is the first input file (it also supports multiple files) shown by **#1 #2 #3**
 - the **mov** it is just the container family :- QuickTime/MP4 family.

```
major_brand       : isom
minor_version     : 512
compatible_brands: isomiso2avc1mp41
encoder           : Lavf62.12.100
```

- isom — ISO Base Media file format, the spec .mp4 is built on.
- compatible_brands: isomiso2avc1mp41 — the file declares compatibility with isom, iso2, avc1 (H.264), and mp41 (MP4 version 1)
- encoder: Lavf62.12.100 — written by FFmpeg's libavformat, version 62.12.100
- **Duration: 00:00:53.33, start: 0.000000, bitrate: 13503 kb/s**

- Duration :- 53sec,33 milliseconds
- bitrate: 13503 kb/s — total bitrate, video + audio combined. ~13.5 Mbps for 53 seconds.

Stream #0:0[0x1](und): Video: h264 (High) (avc1 / 0x31637661), yuv420p(progressive), 2560x1600, 13477 kb/s, 39.41 fps, 330 tbr, 90k tbn (default)

- Stream #0 : 0 — file 0, stream 0 (first stream)
- [0x1] — internal stream ID in the container, hexadecimal
- (und) — language tag is undefined (not set).
- h264 (High) — H.264 codec, High profile. H.264 has profiles: Baseline → Main → High → High 10 etc. High is standard for good quality encodes
- (avc1 / 0x31637661) — the FourCC code. Every codec has a 4-character identifier baked into the container. avc1 is H.264's FourCC. The hex is just the same thing in bytes
- yuv420p(progressive) — color space + scan type. Progressive means full frames, as opposed to interlaced (where frames are split into alternating lines, a relic of old broadcast TV)
- 2560x1600 — your exact monitor resolution
- 13477 kb/s — video stream bitrate specifically
- 39.41 fps — fps
- 330 tbr — **timebase rate**, the internal clock resolution FFmpeg uses to timestamp frames. You don't need to worry about this
- 90k tbn — **ticks per second** of the container's internal timestamp unit. MP4 uses 90,000 ticks/second as its standard. Again, internal bookkeeping

`(default)` — this is the default stream that will play if you don't specify otherwise

```
Stream #0:1[0x2](und): Audio: aac (LC) (mp4a /  
0x6134706D), 48000 Hz, stereo, fltp, 41 kb/s  
(default)
```

- `Stream #0 :1` — file 0, stream 1 (second stream)
- `aac (LC)` — AAC codec, Low Complexity profile. This is the standard AAC variant used everywhere
- `mp4a / 0x6134706D` — FourCC for AAC audio in MP4
- `48000 Hz` — 48,000 samples per second. Standard for video audio (as opposed to 44,100 Hz which is CD/music standard)
- `stereo` — 2 channels
- `fltp` — float planar — this is the internal sample format FFmpeg uses in memory when decoding. Not a property of the file itself, just how FFmpeg holds it internally
- `41 kb/s` — very low audio bitrate

FFMPEG

Command structure

```
ffmpeg [global options] [input options] -i input  
[output options] output
```

The order matters. Options placed before `-i` apply to the input. Options placed after `-i` apply to the output.

Simplest possible Command


```
ffmpeg -i input.mp4 output.mkv
```

FFmpeg sees that you want `.mkv` output and will try to make sensible decisions about everything else automatically.

But here's the critical thing to understand — **FFmpeg will re-encode by default**. It'll decode the video and re-encode it, which takes time and introduces quality loss.

`-c copy` (The star ★)

```
ffmpeg -i input.mp4 -c copy output.mkv
```

-c means codec

copy means copy it so there will be no container change but not the quality loss and all the problems we discussed in the Simplest command section

This is called a remux. The video and audio data is not touched at all — FFmpeg just unpacks the streams from `.mp4` and repacks them into `.mkv`. It's:

- Instant — no encoding work being done
- Lossless — bit-for-bit identical streams
- Zero quality Loss

flags	Meaning
<code>-c copy</code>	Copy all the streams (audio+video)
<code>-c:v copy</code>	Copy video stream Only
<code>-c:a copy</code>	Copy audio stream Only
<code>-c:v libx264</code>	Encode video with H.265
<code>-c:a aac</code>	Encode audio with AAC

`:v` , `:a` these are stream specifiers

`-v` flag

- controls the verbosity of the command
- When we'll run the command for the first time you'll notice that the output is too much so there are some options to control then

Levels are: quiet, panic, fatal, error, warning, info (default), verbose, debug.

```
ffmpeg -v warning -i input.mp4 -c:a acc  
output.mp4a
```

For daily use, `-v warning` is a good sweet spot — it'll only print if something goes wrong.

Reading time

```
• ▶ ffmpeg -i recording_2026-04-29_23.53.40.mp4  
-c copy output.mkv  
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from  
'recording_2026-04-29_23.53.40.mp4':  
Metadata:  
  major_brand      : isom  
  minor_version    : 512  
  compatible_brands: isomiso2avc1mp41  
  encoder          : Lavf62.12.100  
Duration: 00:00:53.33, start: 0.000000, bitrate:  
13503 kb/s  
Stream #0:0[0x1](und): Video: h264 (High) (avc1  
/ 0x31637661), yuv420p(progressive), 2560x1600,
```

13477 kb/s, 39.41 fps, 330 tbr, 90k tbn (default)

Metadata:

handler_name : VideoHandler

Stream #0:1[0x2](und): Audio: aac (LC) (mp4a / 0x6134706D), 48000 Hz, stereo, fltp, 41 kb/s (default)

Metadata:

handler_name : SoundHandler

Stream mapping:

Stream #0:0 → #0:0 (copy)

Stream #0:1 → #0:1 (copy)

Output #0, matroska, to 'output.mkv':

Metadata:

major_brand : isom

minor_version : 512

compatible_brands: isomiso2avc1mp41

encoder : Lavf62.12.101

Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p(progressive), 2560×1600, q=2-31, 13477 kb/s, 39.41 fps, 330 tbr, 1k tbn (default)

Metadata:

handler_name : VideoHandler

Stream #0:1(und): Audio: aac (LC) ([255][0][0][0] / 0x00FF), 48000 Hz, stereo, fltp, 41 kb/s (default)

Metadata:

handler_name : SoundHandler

Press [q] to stop, [?] for help

[out#0/matroska @ 0x560346cb4dc0] video:87584KiB
audio:272KiB subtitle:0KiB other streams:0KiB
global headers:0KiB muxing overhead: 0.060478%

```
frame= 2098 fps=0.0 q=-1.0 Lsize=    87909KiB
time=00:00:53.24 bitrate=13526.1kbits/s speed=
724x elapsed=0:00:00.07
```

- input has the information about the input stream the same information we decoded earlier
- output has the information about the output stream

Output #0, matroska, to 'output.mkv':

- matroska is the original name of the `.mkv` container

Stream #0:0: Video: h264 (High) (avc1 / 0x31637661), yuv420p(progressive), 2560x1600, q=2-31, 13477 kb/s :-

- it the same as the input file as we asked it to copy
- q=2-31 is just the quantizer range for H.264

Audio: aac (LC) ([255][0][0][0] / 0x00FF)

Notice the audio FourCC changed from mp4a / 0x6134706D to [255][0][0][0] / 0x00FF. This is because AAC's FourCC in MP4 containers is mp4a, but in MKV containers AAC doesn't have a named FourCC — MKV uses a raw codec ID instead. The audio data itself is completely identical, just the container-level identifier changed.

```
video:87584KiB audio:272KiB subtitle:0KiB other
streams:0KiB global headers:0KiB muxing overhead:
0.060478%
frame=2098 fps=0.0 q=-1.0 Lsize=87909KiB
time=00:00:53.24 bitrate=13526.1kbits/s speed=724x
elapsed=0:00:00.07
```

- **video:** 87584 KiB — the video stream is ~85 MB
- **audio:** 272 KiB — the audio stream is tiny, ~272 KB
- **muxing** overhead: 0.060478% — the MKV container itself adds almost nothing
- **frame=2098** — 2098 frames in the video total
- **q=-1.0** — confirms no encoding happened (-1 means copy mode)
- **speed=724x** — it processed the video at 724 times realtime. A 53 second video done in 0.07 seconds
- **elapsed=0:00:00.07** — 7 milliseconds. That's how fast a remux is vs an encode

This is the clearest possible demonstration of the container vs codec distinction — changing the container costs almost nothing because the actual data wasn't touched.

Compression Techniques

Two ways to control output size

1. **CRF** :- Constant Rate Factor

- You tell FFmpeg a quality level, and it figures out the bitrate automatically per-frame. File size is not guaranteed but quality is consistent.

2. **CBR** :- Target Bitrate

- You tell FFmpeg a specific bitrate, and it hits that number. File size is predictable but quality may vary.

For most use cases — archiving, sharing, uploading — CRF is what you want. It's how YouTube and Netflix encode their master files too.

0 ←

→ 51

best quality	worst
quality	
(huge file)	(tiny
file)	

For H.264:

- 0 = lossless (enormous)
- 18 = visually lossless (most people can't tell from original)
- 23 = default, good balance
- 28 = noticeably compressed but acceptable
- 51 = unwatchable

The scale is logarithmic - every +6 roughly doubles the file size, every -6 roughly halves it.

```
ffmpeg -i recording_2026-04-29_23.53.40.mp4 -c:v  
libx264 -crf 23 -c:a copy output_compressed.mp4
```

Part	Meaning
-c:v libx264	Encode video with H.264 encoder
-crf 23	Quality level 23
-c:a copy	Don't touch the audio, copy it as-is
output_compressed.mp4	Stay in MP4 container

The Three types of frames

I-Frame

A complete, self-contained image. No reference to any other frame. If you've ever seeked to a random point in a video and it snapped to a slightly different timestamp — that's because it jumped to the nearest keyframe. You cannot decode any other frame type without first having a keyframe.

P-Frame

Only stores the difference from the **previous frame**. If a ball moves across a static background, the P-frame only stores the ball's new position and what the background looks like where the ball used to be. Everything unchanged is just referenced from the previous frame. Much smaller than an I-frame.

B-Frame

References both the previous AND the next frame. Even more efficient than P-frames because it can borrow from both directions. This is why `keyint=250` and `bframes=3` appeared in your encode — the encoder was allowed to look ahead and behind.

```
I B B P B B P B B P B B I
↑                               ↑
keyframe                     next keyframe
```

Generation loss :- critical concept

Every time you **decode and re-encode** a video, you lose quality. Even at the same CRF. This is because:

1. The encoder makes lossy approximations

2. Re-encoding approximates those approximations again
3. Errors accumulate

This is called generation loss and it's why:

- You should always encode from the highest quality source available
- You should avoid re-encoding an already-compressed file multiple times
- The remux (-c copy) exists — when you only need to change the container, never re-encode

Codec generations

Codecs have evolved in generations. Each generation *roughly doubles* the compression efficiency of the previous one at the same quality level:

MPEG-2 (1995) → H.264/AVC (2003) → H.265/HEVC (2013) → AV1 (2018)

"Doubles the compression efficiency" means: same visual quality at half the file size, or better quality at the same file size.

Codec	Encoder in FFmpeg	Generation	Compression	Encode Speed	Hardware Support	Patent Status
H.264	libx264	3rd	Baseline	Very fast	Universal	Patented
H.265	libx265	4th	~50% better than H.264	Slow	Wide	Patented
VP9	libvpx-vp9	4th	Similar to H.265	Very slow	Moderate	Open/Free
AV1	libsvtav1	5th	~50% better than H.265	Slow-Moderate	Growing	Open/Free

"Better compression" always means same quality, smaller file — not worse quality.

Audio Codecs

Codec	Encoder in FFmpeg	Quality/Size	Use Case
MP3	<code>libmp3lame</code>	Decent	Legacy, universal compatibility
AAC	<code>aac</code> or <code>libfdk_aac</code>	Better than MP3 at same bitrate	Standard for MP4/streaming
Opus	<code>libopus</code>	Best efficiency at low bitrates	WebM, Discord, VoIP
FLAC	<code>flac</code>	Lossless	Archiving, music production
PCM	<code>pcm_s16le</code> etc.	Uncompressed	Raw, no quality loss at all

which codec for which container

.mp4

- Video: **H.264** (universal compatibility) or **H.265** (better compression, less compatible with old devices)
- Audio: **AAC**
- Don't use: VP9, AV1 (poor/no support in MP4), Opus (not supported)
- Best for: sharing, uploading, broad device compatibility

.mkv

- Video: **anything** — H.264, H.265, AV1, VP9
- Audio: **anything** — AAC, Opus, FLAC, MP3
- Don't use: nothing, MKV accepts everything

- Best for: local archiving, storing multiple audio/subtitle tracks, future-proofing

.webm

- Video: **VP9** (widely supported) or **AV1** (better compression, newer)
- Audio: **Opus** (preferred) or Vorbis
- Don't use: H.264, H.265 (not supported at all)
- Best for: web delivery, open-source pipelines, browser-native playback

.avi

- Just don't use it for new work. It's a 1992 format with no modern features. You'll encounter it when dealing with old files, but never create new ones.

So what should YOU use for your screen recordings?

You have a 2560x1600 screen recording. Here's what I'd actually recommend based on your use case:

For sharing / uploading:

H.264 + AAC in .mp4

Maximum compatibility. Every device, browser, and platform plays it.

For local archiving:

H.265 + AAC in .mkv

~50% smaller than H.264 at the same quality. MKV in case you want to add subtitle tracks later.

For maximum compression, future-proof archiving:

AV1 + Opus in .mkv

Smallest file at same quality. Encoding is slower but you only do it once.

Container	Best Quality Codec	Best Compression Codec	Best Audio
.mp4	H.264	H.265	AAC
.mkv	AV1	AV1	Opus
.webm	VP9	AV1	Opus